Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Tanay Malvankar		Subject: ASD&D
T.E. - T1 Batch: T13 Sem: V	Roll No: 58	Date: 30/07/26

Experiment 2

Aim: To Containerize Application using Docker

LO Mapped: LO4

LO4: Containerization, orchestrate and deploy applications using Docker, Compose, Kubernetes and Cloud infrastructure tools.

Theory:

Docker is a containerization platform that helps developers build, package, and deploy software in a consistent environment. Instead of installing an application and its dependencies separately on every system, Docker packages the required components into containers. This makes applications easier to move and run across development, testing, and production environments.

Containerization refers to the technique of placing an application, together with its runtime, libraries, configuration, and other dependencies, inside a container. Since the necessary components are packaged together, the application can operate in a similar manner regardless of the underlying environment.

A **Docker image** is a packaged, read-only blueprint used for

creating containers. It contains the files, application code, libraries, runtime components, and configuration required by an application. Docker images are commonly created from instructions specified in a Dockerfile.

A **Docker container** is an executable instance created from a Docker image. It provides an isolated environment in which an application and its processes can run. A single image can be used to create multiple containers, with each container operating independently.

Docker Image vs Docker Container

Docker Image	**Docker Container**
Acts as a blueprint for an application	Represents an active instance of an image
Contains application files and dependencies	Uses the image contents to execute the application
Generally read-only	Has a writable container layer
Usually built using a Dockerfile	Commonly created using docker run
Does not run an application by itself	Provides the environment in which the application runs

Benefits of Docker

- Applications can be packaged and moved easily between different environments.
- Containers provide process and application-level isolation.
- Application deployment becomes simpler because dependencies are packaged with the application.
- Containers generally require fewer resources than traditional virtual machines.
- Development, testing, and deployment workflows can be made faster and more consistent.

1. Docker Installation and Verification

1. Download and install Docker Desktop from the official Docker website.
2. Complete the installation by following the instructions provided by the setup wizard.
3. Restart the system if the installation process is requested.
4. Launch Docker Desktop and allow the Docker engine to start.
5. Open a terminal such as Command Prompt, PowerShell, or Ubuntu/WSL and verify the Docker installation using:

docker --version

6. Display the Docker client and server configuration using:

docker info

7. Run the sample Hello World container to confirm that Docker can successfully communicate with the Docker engine and create a container:

docker run hello-world

Commands Executed:

1. Check Docker Version

docker –version

```
C:\Users\15L>docker --version
Docker version 29.2.1, build a5c7197
```

2. Search for Docker Images

docker search ubuntu

This command searches Docker Hub for images related to **Ubuntu**.

```
C:\Users\15L>docker search ubuntu
NAME                                DESCRIPTION                                     STARS     OFFICIAL
ubuntu                              Ubuntu is a Debian-based Linux operating sys…   17859     [OK]
ubuntu/squid                        Squid is a caching proxy for the Web. Long-t…   129
ubuntu/nginx                        Nginx, a high-performance reverse proxy & we…   141
ubuntu/bind9                        BIND 9 is a very flexible, full-featured DNS…   120
ubuntu/kafka                        Apache Kafka, a distributed event streaming …   62
ubuntu/zookeeper                    ZooKeeper maintains configuration informatio…   15
ubuntu/apache2                      Apache, a secure & extensible open-source HT…   104
ubuntu/prometheus                   Prometheus is a systems and service monitori…   80
ubuntu/jre                          Chiseled Java runtime based on Ubuntu. Long-…   25
ubuntu/dotnet-aspnet                Chiselled Ubuntu runtime image for ASP.NET a…   26
ubuntu/python                       A chiselled Ubuntu rock with the Python runt…   33
ubuntu/mysql                        MySQL open source fast, stable, multi-thread…   74
ubuntu/postgres                     PostgreSQL is an open source object-relation…   45
ubuntu/redis                        Redis, an open source key-value store. Long-…   26
ubuntu/dotnet-runtime               Chiselled Ubuntu runtime image for .NET apps…   24
ubuntu/dotnet-deps                  Chiselled Ubuntu for self-contained .NET & A…   16
ubuntu/grafana                      Grafana, a feature rich metrics dashboard & …   12
ubuntu/memcached                    Memcached, in-memory keyvalue store for smal…   7
ubuntu/jdk                          Ubuntu rock for OpenJDK                         0
ubuntu/rabbitmq                     Ubuntu-based RabbitMQ image - an open source…   0
ubuntu/mimir                        Ubuntu ROCK for Mimir, a horizontally scalab…   0
ubuntu/loki                         Grafana Loki, a log aggregation system like …   2
ubuntu/traefik                      Ubuntu ROCK for Traefik, a modern HTTP rever…   3
ubuntu/opentelemetry-collector      This is a rock for the OpenTelemetry Collect…   0
ubuntu/alertmanager                 Ubuntu Rock for Alertmanager. It handles ale…   1
```

3. Search with a Filter

docker search ubuntu --filter stars=100

This command displays Ubuntu images that have **100 or more stars** on Docker Hub.

```
C:\Users\15L>docker search ubuntu --filter stars=100
NAME              DESCRIPTION                                     STARS     OFFICIAL
ubuntu            Ubuntu is a Debian-based Linux operating sys…   17859     [OK]
ubuntu/squid      Squid is a caching proxy for the Web. Long-t…   129
ubuntu/nginx      Nginx, a high-performance reverse proxy & we…   141
ubuntu/bind9      BIND 9 is a very flexible, full-featured DNS…   120
ubuntu/apache2    Apache, a secure & extensible open-source HT…   104
```

4. Download an Ubuntu Image

docker pull ubuntu

This command downloads the latest Ubuntu image from Docker Hub to the local system.

```
C:\Users\15L>docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
Digest: sha256:3131b4cc82a783df6c9df078f86e01819a13594b865c2cad47bd1bca2b7063bb
Status: Image is up to date for ubuntu:latest
docker.io/library/ubuntu:latest
```

5. List Downloaded Images

docker images

```
C:\Users\15L>docker images

IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   ef54e839ef54       25.9kB         9.52kB    U
nginx:latest         5a88c9c45479        241MB           66MB    U
redis:latest         1c054d54ecd1        204MB         55.3MB    U
ubuntu:latest        3131b4cc82a7        161MB         45.3MB    U
```

6. Run Ubuntu Container

docker run -it ubuntu bash

```
C:\Users\15L>docker run -it ubuntu bash
root@654f898394b6:/#
```

7. Check Ubuntu Version (Inside Container)

cat /etc/os-release

```
C:\Users\15L>docker run -it ubuntu bash
root@654f898394b6:/# cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04 LTS (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
```

8. Exit the Container

exit

```
root@654f898394b6:/# exit
exit
```

9. List All Containers

docker ps -a

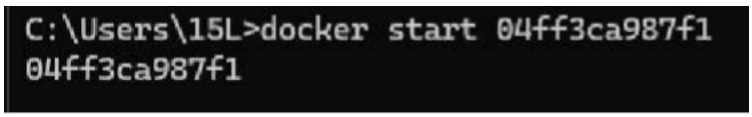


```
C:\Users\15L>docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS                          PORTS
          NAMES
654f898394b6   ubuntu         "bash"                   About a minute ago   Exited (0) 34 seconds ago
          vigilant_albattani
f7b09cfcb56e   nginx          "/docker-entrypoint.…"   18 hours ago         Created
          practice
e41f9ed60c3c   nginx          "/docker-entrypoint.…"   18 hours ago         Exited (255) 28 minutes ago     0.0.0.0:8080->
80/tcp    mynginx
1008bc0dcc79   ubuntu         "/bin/bash"              19 hours ago         Exited (0) 19 hours ago
          mystifying_wilson
b80193fd8c86   ubuntu         "/bin/bash"              19 hours ago         Exited (0) 19 hours ago
          jolly_hofstadter
ce280af76907   ubuntu         "/bin/bash"              19 hours ago         Exited (0) 19 hours ago
          sharp_pasteur
04ff3ca987f1   hello-world    "/hello"                 4 months ago         Exited (0) 4 months ago
          focused_agnesi
506fbcc93880   redis:latest   "docker-entrypoint.s…"   4 months ago         Exited (255) 19 hours ago       6379/tcp
          distracted_babbage
d3e8d53b8e82   redis          "docker-entrypoint.s…"   4 months ago         Exited (255) 19 hours ago       6379/tcp
          competent_rosalind
```

10. Start the Stopped Container

First note the **Container ID** from docker ps -a.

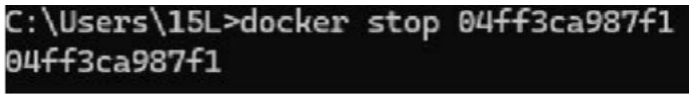
docker start <container_id>



```
C:\Users\15L>docker start 04ff3ca987f1
04ff3ca987f1
```

11. Stop the Running Container

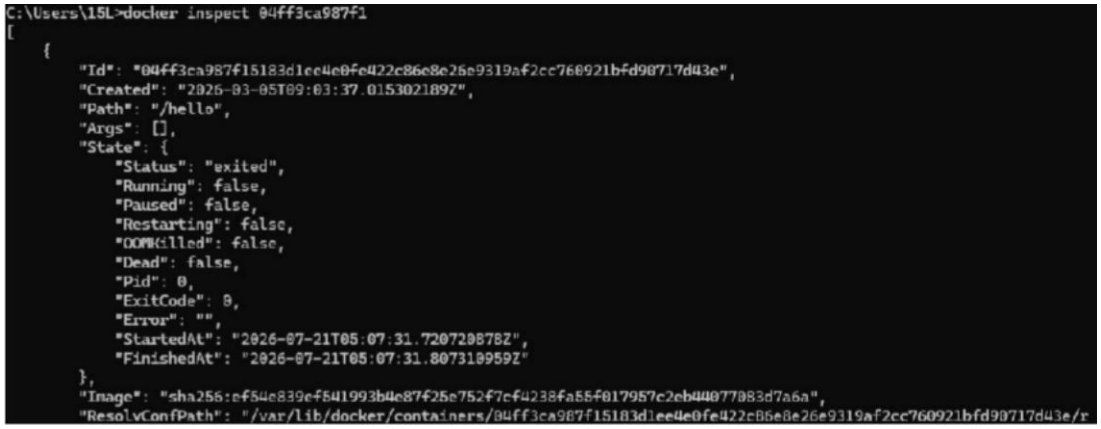
docker stop <container_id>



```
C:\Users\15L>docker stop 04ff3ca987f1
04ff3ca987f1
```

12. Inspect a Container

docker inspect <container_id>



```
C:\Users\15L>docker inspect 04ff3ca987f1
[
    {
        "Id": "04ff3ca987f15183d1ee4e0fe422c86e8e26e9319af2cc760921bfd90717d43e",
        "Created": "2026-03-05T09:03:37.015302189Z",
        "Path": "/hello",
        "Args": [],
        "State": {
            "Status": "exited",
            "Running": false,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 0,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2026-07-21T05:07:31.720720878Z",
            "FinishedAt": "2026-07-21T05:07:31.807310959Z"
        },
        "Image": "sha256:ef54e839ef541993b4e87f25e752f7cf4238fa55f017957c2eb44077083d7a6a",
        "ResolvConfPath": "/var/lib/docker/containers/04ff3ca987f15183d1ee4e0fe422c86e8e26e9319af2cc760921bfd90717d43e/r
```

13. View Running Containers

docker ps

```
C:\Users\15L>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

14. Remove a Container

docker rm <container_id>

```
C:\Users\15L>docker rm 04ff3ca987f1
04ff3ca987f1
```

15. Remove the Ubuntu Image

docker rmi ubuntu

```
C:\Users\15L>docker rmi ubuntu
Untagged: ubuntu:latest
Deleted: sha256:3131b4cc82a783df6c9df078f86e01819a13594b865c2cad47bd1bca2b7063bb
```

Conclusion:

This experiment helped me understand the fundamentals of Docker and containerization. I learned how to install Docker, work with Docker images and containers, and use essential commands such as docker pull, docker run, docker ps, docker stop, and docker rm. It also gave me practical knowledge of how Docker simplifies application deployment, provides isolation, and ensures consistency across different environments.